

Introducción a la teoría de la navegación autónoma

Explicación e implementación de algoritmos SLAM con optimización gráfica

Aarón Flores Alberca

Universidad Nacional de Ingeniería. Facultad de Ciencias.
Escuela Profesional de Ciencia de la Computación

28 de febrero de 2025

Contenido

- 1 Introducción
- 2 Marco teórico
- 3 Explotando la Dispersión
- 4 Algoritmos avanzados para SLAM
- 5 Conclusiones
- 6 Referencias

- 1 Introducción
- 2 Marco teórico
- 3 Explotando la Dispersión
- 4 Algoritmos avanzados para SLAM
- 5 Conclusiones
- 6 Referencias

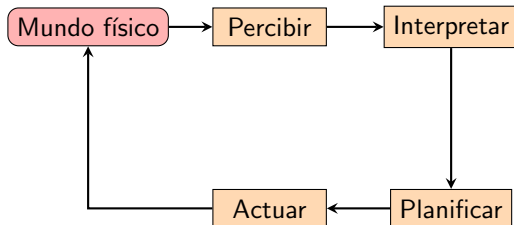
Se entiende a la navegación autónoma como la capacidad de un sistema, generalmente un vehículo o robot, para desplazarse sin intervención humana en un entorno ya sea conocido o desconocido. Para alcanzar dicha independencia se tienen dos opciones:

- **Heurísticas matemáticas:** Se designa un conjunto de reglas generales y preestablecidas de interacción entre el móvil y su entorno para la completitud de ciertas metas.
- **Optimización:** El agente o sistema involucrado se adecúa a su entorno capturando e interpretando datos de este para la completitud de sus tareas designadas, por lo mismo requiere de muchísima información.

Introducción (cont.)

Aunque la tarea de construir dichos modelos sea demasiado compleja, muchísimas compañías actuales se encuentran interesados en construir vehículos con dichas capacidades como Tesla, Google e incluso Nvidia.

Se puede estructurar el funcionamiento de sistemas autónomos como el siguiente bucle:



Un componente fundamental de este ciclo es la interpretación del entorno, que implica la localización del robot y el mapeo del entorno.

Sensores en navegación autónoma

Los robots autónomos utilizan diversos sensores para percibir el entorno:

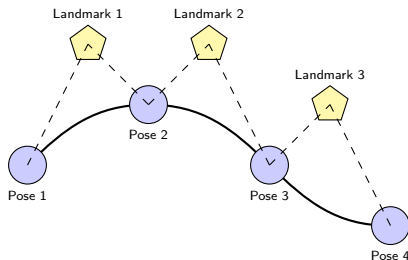
- **LiDAR (Light Detection and Ranging):** Emite pulsos láser que rebotan en superficies y regresan, permitiendo medir distancias con precisión y crear mapas 3D del entorno.
- **IMU (Unidad de Medición Inercial):** Combina acelerómetros, giroscopios y magnetómetros para medir aceleración lineal y velocidad angular, proporcionando información sobre el movimiento del robot.
- **Cámaras:** Proporcionan información visual del entorno, utilizadas para detección de objetos, líneas y características distintivas.
- **Encoders:** Miden el desplazamiento de las ruedas, permitiendo estimar el movimiento del robot (odometría).

La fusión de estos datos sensoriales permite construir una representación coherente del entorno y de la posición del robot mediante una técnica de estimación de posición que calcula el movimiento realizado llamado **odometría**. Sin embargo, la odometría acumula errores con el tiempo debido a deslizamientos, imperfecciones en las ruedas y superficies irregulares.

El problema SLAM

SLAM (Simultaneous Localization and Mapping) es uno de los problemas fundamentales en robótica móvil.

- **Problema:** Un robot explora un entorno desconocido, debe construir un mapa y, simultáneamente, localizarse dentro de él.
- **Desafío:** Para construir un mapa preciso se necesita conocer la posición del robot, pero para conocer la posición se necesita un mapa fiable.
- **Incertidumbre:** Tanto el mapa como la posición del robot contienen errores que se propagan y acumulan.



Matrices de Transformación en Navegación Autónoma

- **Transformación Rígida en 3D:** Representada mediante matrices 4×4 en $SE(3)$

$$T = \begin{bmatrix} R & t \\ 0_{1 \times 3} & 1 \end{bmatrix}$$

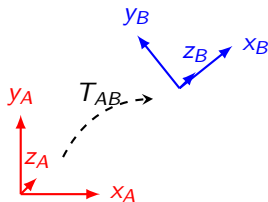
donde $R \in SO(3)$ es la matriz de rotación y $t \in \mathbb{R}^3$ es el vector de traslación

- **Composición de transformaciones:** Multiplicación matricial

$$T_{AB} \cdot T_{BC} = T_{AC}$$

- **Transformación inversa:**

$$T^{-1} = \begin{bmatrix} R^T & -R^T t \\ 0_{1 \times 3} & 1 \end{bmatrix}$$



El algoritmo ICP

El algoritmo **ICP (Iterative Closest Point)** es fundamental en SLAM para alinear nubes de puntos y estimar el movimiento del robot:

- **Objetivo:** Encontrar la transformación (rotación y traslación) que mejor alinea dos conjuntos de puntos.
- **Proceso iterativo:**
 - 1 Asociar puntos entre ambas nubes (correspondencia)
 - 2 Estimar la transformación que minimiza la distancia entre puntos correspondientes
 - 3 Aplicar la transformación y repetir hasta converger
- **Aplicaciones:** Registro de scans LiDAR consecutivos, construcción de mapas 3D, estimación de odometría.

ICP es susceptible a mínimos locales, por lo que requiere una buena inicialización. Es comúnmente utilizado en combinación con otros métodos de estimación de movimiento.

Contenido

- 1 Introducción
- 2 **Marco teórico**
- 3 Explotando la Dispersión
- 4 Algoritmos avanzados para SLAM
- 5 Conclusiones
- 6 Referencias

Conceptos de cálculo de probabilidades

Se denomina **variable aleatoria** (v.a) X a una función real que asigna un valor numérico a resultados posibles de eventos aleatorios, matemáticamente X se define como una función $X : \Omega \rightarrow E \subset \mathbf{R}$ donde Ω es un espacio muestral que determina todos los posibles valores que puede tomar dicha variable X y $E \subset \mathbf{R}$ como la representación cuantificada de dichos valores aleatorios.

De este concepto podemos tomar dos acercamientos diferentes referentes a variables aleatorias:

- **V.A discretas:** Se dice que una v.a es discreta si toma valores en un conjunto numerable (finito o infinito numerable). Ejemplo: Número de caras al lanzar 3 veces una moneda o cantidad de turistas visitantes a un museo.
- **V.A continuas:** Se dice que una v.a es continua si puede tomar cualquier valor dentro de un intervalo de números reales. Ejemplo: Medición de la presión en diferentes camaras de un submarino o la estimación del movimiento de un móvil mediante sensores.

Conceptos de cálculo de probabilidades (cont.)

Para esta presentación nos concentraremos en específico con las v.a continuas, con ellas podemos encontrarnos con los siguientes conceptos:

- **Función de densidad de probabilidad:** En el contexto de SLAM, la PDF más común es la distribución Gaussiana multivariada:

$$\mathcal{N}(x; \mu, \Sigma) = p(x) = \frac{1}{\sqrt{|2\pi\Sigma|}} \exp\left(-\frac{1}{2}\|x - \mu\|_{\Sigma}^2\right)$$

donde μ es el vector de medias, Σ es la matriz de covarianza y $\|x - \mu\|_{\Sigma}^2$ representa la distancia de Mahalanobis de los valores x con respecto a la distribución de probabilidad.

- **Modelación de medición:** Es conveniente el modelamiento de las mediciones hechas con incertidumbres con un ruido de tipo Gaussiano de media cero y matriz de covarianza R . Definimos dicha estimación como un valor z :

$$z = h(x, l) + \eta$$

donde $h(\cdot)$ se le denomina una función de predicción de medidad y el ruido η el modelo de incertidumbre declarado anteriormente.

Conceptos de cálculo de probabilidades (cont.)

Es de estas inferencias de donde se llega a la siguiente función de probabilidad condicional adecuada para la medición estimada z :

$$\mathcal{N}(z; h(x, l), R) = p(z|x, l) = \frac{1}{\sqrt{|2\pi R|}} \exp\left(-\frac{1}{2}\|h(x, l) - z\|_R^2\right)$$

En un modelo probabilístico de movimiento, $g(\cdot)$ representa nuestra función de predicción, u_t es una acción conocida de movimiento y Q es su matriz de covarianza asociada. Estos elementos conforman nuestra función de probabilidad.

$$\mathcal{N}(x_{t+1}; g(x_t, u_t), Q) = p(x_{t+1}|x_t, u_t) = \frac{1}{\sqrt{|2\pi Q|}} \exp\left(-\frac{1}{2}\|g(x_t, u_t) - x_{t+1}\|_Q^2\right)$$

Nos topamos con un problema importante, ya que ambas probabilidades fundamentales están siendo expresadas mediante funciones de densidad diferentes aunque iguales en esencia. De estas complicaciones, introducimos el concepto de un factor graph como modelo gráfico probabilístico para la resolución de estas estimaciones.

Factor Graphs

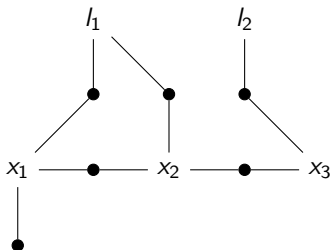
Un factor graph es un grafo bipartito $F = (U, V, E)$ con:

- Nodos variables $\theta_j \in V$ (poses del robot, landmarks)
- Nodos factores $\phi_i \in U$ (mediciones, restricciones)
- Aristas $e_{ij} \in E$ que conectan factores con las variables de las que dependen

A partir de dicho modelo gráfico se obtiene la función de probabilidad total como la productoria de todos los nodos factores.

$$\phi(X) = \prod_i \phi_i(X_i)$$

donde X_i son las variables conectadas al factor ϕ_i .



Factor Graphs en SLAM

Los factor graphs ofrecen una representación natural para el problema SLAM:

- **Variables (X):** Son variables aleatorias almacenadas en la memoria del móvil obtenidas de sus sensores adheridos, con los que se les permite tener una estimación del entorno y de su posición en él.
 - Poses del robot $x_1, x_2, \dots, x_n$
 - Posiciones de landmarks $l_1, l_2, \dots, l_m$
- **Factores:** Representan restricciones probabilísticas sobre las variables mediante funciones de probabilidad para modelar características como mediciones y movimiento.

$$\begin{aligned}\phi_i(X_i) &= \frac{1}{\sqrt{|2\pi\Sigma_i|}} \exp \left\{ -\frac{1}{2} \|h_i(X_i) - z_i\|_{\Sigma_i}^2 \right\} \\ &\propto \exp \left\{ -\frac{1}{2} \|h_i(X_i) - z_i\|_{\Sigma_i}^2 \right\}\end{aligned}$$

Inferencia MAP - Máximo a posteriori

La inferencia MAP busca la configuración más probable de las variables dado los datos observados:

$$X^{MAP} = \arg \max_X \phi(X) = \arg \max_X \prod_i \phi_i(X_i)$$

Podemos convertir nuestra inferencia inicial de la siguiente manera:

$$\begin{aligned} X^{MAP} &\propto \arg \max_X \exp \left\{ \sum_i -\frac{1}{2} \|h_i(X_i) - z_i\|_{\Sigma_i}^2 \right\} \\ &\propto \arg \min_X \sum_i \|h_i(X_i) - z_i\|_{\Sigma_i}^2 \end{aligned}$$

Así, la inferencia MAP se transforma en un problema de mínimos cuadrados ponderados, donde cada término representa la discrepancia entre la predicción del modelo $h_i(X_i)$ y la medición z_i , ponderada por la incertidumbre Σ_i^{-1} .

Linearización

Podemos linealizar nuestras funciones de medición $h_i(\cdot)$ dentro de una simple expansión de una serie de Taylor como la siguiente:

$$h_i(X_i) = h_i(X_i^0 + \Delta_i) \approx h_i(X_i^0) + H_i \Delta_i$$

Donde X_i^0 es un estado inicial de nuestras estimaciones de valores X_i , Δ_i es la diferencia entre dicho estado inicial y el estado a evaluar $X_i - X_i^0$ y finalmente adaptamos a un jacobiano de medición como H_i con la siguiente definición:

$$H_i = \left. \frac{\partial h_i(X_i)}{\partial X_i} \right|_{X_i^0}$$

Substituimos nuestra aproximación realizada dentro de nuestra anterior conceptualización de MAP para cambiar nuestro enfoque de variables que maximizan a un vector de cambio que maximiza:

$$\begin{aligned} \Delta^* &= \arg \min_{\Delta} \sum_i \|h_i(X_i^0) + H_i \Delta_i - z_i\|_{\Sigma_i}^2 \\ &= \arg \min_{\Delta} \sum_i \|H_i \Delta_i - \{z_i - h_i(X_i^0)\}\|_{\Sigma_i}^2 \end{aligned}$$

Linearización (cont.)

Nos encontramos sometidos a una distancia de Mahalanobis, lo que no permite un desarrollo computacionalmente simple, para esto recordemos que nuestra matriz de covarianza es una matriz semidefinida positiva con lo cual se nos permite encontrar una matriz raíz cuadrada a partir de algún método de factorización como LU.

$$\|e\|_{\Sigma}^2 = e^T \Sigma^{-1} e = \left(\Sigma^{-1/2} e\right)^T \left(\Sigma^{-1/2} e\right) = \|\Sigma^{-1/2} e\|_2^2$$

Si considerasemos a nuestro error $e = H_i \Delta_i - \{z_i - h_i(X_i^0)\}$ entonces llegamos a los siguientes conceptos:

$$\begin{aligned} A_i &= \Sigma_i^{-1/2} H_i \\ b_i &= \Sigma_i^{-1/2} (z_i - h_i(X_i^0)) \end{aligned}$$

Reemplazándolo en nuestra problemática inicial nos daremos cuenta de algo interesante:

$$\begin{aligned} \Delta^* &= \arg \min_{\Delta} \sum_i \|A_i \Delta_i - b_i\|_2^2 \\ &= \arg \min_{\Delta} \|A \Delta - b\|_2^2 \end{aligned}$$

Llegamos a un sistema de ecuaciones lineales

Método del descenso de gradiente

Con esta nueva conversión podemos encontrarnos con distintos métodos de resolución que serían una factorización QR, Gauss-Newton, Levenberg-Marquardt, etc. Abarcaremos el más 'simple' mediante el descenso de gradiente.

Damos el siguiente modelo iterativo para encontrar una solución a nuestro problema:

$$X^{t+1} = X^t + \Delta$$

Comenzamos definiendo a nuestra función costo como la siguiente:

$$g(X) = \sum_i \|h_i(X_i) - z_i\|_{\Sigma_i}^2 \approx \|A(X - X^t) - b\|_2^2$$

Finalmente calculamos a Δ mediante esta aproximación:

$$\Delta = -\alpha \nabla g(X)|_{X=X^t} = -\alpha (-2A^T b) = 2\alpha A^T b$$

$$X^{t+1} = X^t + 2\alpha A^T b$$

donde α es la tasa de convergencia, lamentablemente se vuelve ligeramente lento al encontrar mínimos.

Contenido

- 1 Introducción
- 2 Marco teórico
- 3 Explotando la Dispersión
- 4 Algoritmos avanzados para SLAM
- 5 Conclusiones
- 6 Referencias

Estructura Dispersa en SLAM

La dispersión es una característica fundamental en SLAM que refleja la naturaleza física de las mediciones robóticas: En problemas SLAM, la matriz inicial A y la de información $\Lambda = A^T A$ presenta una alta dispersión debido a las siguientes razones:

- Cada medición relaciona solo unas pocas variables (generalmente solo dos)
- Poses consecutivas del robot están conectadas formando una cadena
- Los landmarks solo son observados desde un pequeño subconjunto de poses

En la siguiente matriz podemos encontrarnos con la representación del sistema de ecuaciones lineales de nuestro ejemplo inicial:

$$[A|b] = \begin{array}{c} \phi_1 \\ \phi_2 \\ \phi_3 \\ \phi_4 \\ \phi_5 \\ \phi_6 \\ \phi_7 \\ \phi_8 \\ \phi_9 \end{array} \left[\begin{array}{cccccc|c} \delta l_1 & \delta l_2 & \delta x_1 & \delta x_2 & \delta x_3 & b \\ & & A_{13} & & & b_1 \\ & & A_{23} & A_{24} & & b_2 \\ & & & A_{34} & A_{35} & b_3 \\ A_{41} & & & & & b_4 \\ & A_{52} & & & & b_5 \\ & & A_{63} & & & b_6 \\ A_{71} & & A_{73} & & & b_7 \\ A_{81} & & & A_{84} & & b_8 \\ & A_{92} & & & A_{95} & b_9 \end{array} \right]$$

El reconocimiento y explotación de esta dispersión permite transformar problemas aparentemente intratables en computacionalmente viables.

Beneficios Computacionales de la Dispersión

Esta estructura dispersa tiene un impacto directo en la representación computacional:

- **Eficiencia en almacenamiento:** Solo se almacenan los elementos no-cero mediante formatos especializados como CSR o CSC, reduciendo el consumo de memoria en más del 99 % para problemas grandes.
- **Operaciones matriciales optimizadas:** Las operaciones básicas se vuelven significativamente más eficientes.
 - Multiplicación matriz-vector: $O(k)$ en lugar de $O(n^2)$
 - Factorización de la matriz: $O(n)$ en vez de $O(n^3)$ para muchas configuraciones de SLAM
- **Algoritmos adaptados:** Se pueden utilizar técnicas especializadas como el ordenamiento de variables, la actualización incremental y las implementaciones distribuidas y paralelas para resolver ese tipo de problemas a gran escala.

Estas ventajas permiten resolver sistemas con millones de variables, haciendo posible aplicaciones de SLAM en tiempo real y a gran escala.

Contenido

- 1 Introducción
- 2 Marco teórico
- 3 Explotando la Dispersión
- 4 Algoritmos avanzados para SLAM**
- 5 Conclusiones
- 6 Referencias

KISS-ICP: Simplicidad efectiva con adaptación de velocidad para predicción de movimiento, ponderación por Mahalanobis y rechazo inteligente de outliers. Destaca por su rendimiento superior con implementación de menos de 500 líneas de código. Ideal para sistemas con recursos limitados.

CT-ICP: Corrección de distorsión temporal mediante modelado de trayectoria con B-splines, desdistorsión de puntos según timestamp y adaptación del intervalo de control según velocidad. Especialmente efectivo en vehículos de alta velocidad, drones y robots ágiles donde la distorsión es significativa.

GLIM: Fusión LiDAR-IMU en factor graphs con optimización conjunta de poses y velocidades. Integra factores complementarios (LiDAR, IMU, loop closure) y calibración en línea. Proporciona robustez en entornos con poca geometría o movimiento rápido, manteniendo la consistencia global del mapa.

Contenido

- 1 Introducción
- 2 Marco teórico
- 3 Explotando la Dispersión
- 4 Algoritmos avanzados para SLAM
- 5 Conclusiones**
- 6 Referencias

- Los factor graphs proporcionan un marco unificado para representar y resolver problemas de SLAM.
- La estructura dispersa de los problemas SLAM permite algoritmos eficientes.
- La optimización no lineal sobre manifolds es clave para manejar correctamente rotaciones y transformaciones.
- Los algoritmos incrementales permiten aplicaciones en tiempo real.
- El campo continúa evolucionando hacia sistemas más robustos y aplicaciones en entornos complejos.

Los factor graphs se han convertido en el enfoque dominante para SLAM, proporcionando una base teórica sólida y soluciones prácticas para la navegación autónoma.

Contenido

- 1 Introducción
- 2 Marco teórico
- 3 Explotando la Dispersión
- 4 Algoritmos avanzados para SLAM
- 5 Conclusiones
- 6 Referencias**

- [1] Factor Graphs for Robot Perception. (s. f.). Now Foundations And Trends Books — IEEE Xplore. <https://ieeexplore.ieee.org/document/8187520>
- [2] Vizzo, I., Guadagnino, T., Mersch, B., Wiesmann, L., Behley, J., Stachniss, C. (2023). KISS-ICP: In Defense of Point-to-Point ICP – Simple, Accurate, and Robust Registration If Done the Right Way. IEEE Robotics And Automation Letters, 8(2), 1029-1036. <https://doi.org/10.1109/lra.2023.3236571>
- [3] Dellenbach, P., Deschaud, J., Jacquet, B., Goulette, F. (2021, 27 septiembre). CT-ICP: Real-time Elastic LiDAR Odometry with Loop Closure. arXiv.org. <https://arxiv.org/abs/2109.12979>
- [4] Koide, K., Yokozuka, M., Oishi, S., Banno, A. (2024). GLIM: 3D range-inertial localization and mapping with GPU-accelerated scan matching factors. Robotics And Autonomous Systems, 179, 104750. <https://doi.org/10.1016/j.robot.2024.104750>